**CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.**

**Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System**

**Name: ANKIREDDY ABHISHEK REDDY**

**Reg.no: 192424192**

**Course: Design and Analysis of Algorithms**

**Course code: CSA0601**

## TASK 1 – DESIGN THE ALGORITHM

### Introduction

The university maintains a large database containing student records. Every student has a unique registration number. Faculty members and administrative staff need to search for a particular registration number and retrieve the corresponding student record.

The number of records can increase from 1,000 to 10,000, 100,000 and even 1,000,000 or more. Therefore, using a simple searching method may not always be efficient when the database becomes very large.

For this problem, I selected three different searching strategies:

### 1. Linear Search

### 2. Recursive Binary Search

### 3. Hash Search

I selected these three because they work in different ways. Linear search checks records sequentially, binary search reduces the search area by dividing it into halves, and hashing tries to directly locate the required record using a hash value.

### 1. Linear Search

Linear search is the simplest searching technique. In this method, I start from the first record and compare its registration number with the required registration number.

If the registration number matches, the search is successful. If it does not match, I move to the next record and continue the same process.

For example, suppose the records are:

1021 1050 1075 1100  1140

If I want to find 1100, I compare:

1021 → No

1050 → No

1075 → No

1100 → Found

Therefore, four comparisons are required.

### Type

Linear search is an iterative/sequential approach.

### Preprocessing

No preprocessing is required. The records do not need to be sorted.

### Pseudocode

LinearSearch (A, n, key)

1\. Start from the first record

2\. Compare A[i] with key

3\. If A[i] = key

return "Found"

4\. Move to the next record

5\. Continue until all records are checked

6\. Return "Not Found"

### Why I selected Linear Search

I selected linear search because it is easy to understand and implement. It can also work directly on unsorted records. It provides a useful basic method against which the other algorithms can be compared.

Its main disadvantage is that it may have to check many records when the database is large.

### 2. Recursive Binary Search

Binary search is a more efficient searching method, but it requires the registration numbers to be sorted.

Instead of checking every record, binary search checks the middle record.

For example:

1001 1010 1030 1050 1070 1090 1110

↑

middle

If I want to find 1090, I compare it with 1050.

Since 1090 > 1050, I know that the required value must be on the right side. Therefore, the left half can be ignored.

The same process is repeated on the remaining half.

### Type

Binary search follows the divide-and-conquer strategy. I am using the recursive version for this assignment.

### Preprocessing

The records must be sorted before applying binary search.

### Pseudocode

BinarySearch(A, low, high, key)

1\. If low > high

return "Not Found"

2\. mid = (low + high) / 2

3\. If A[mid] == key

return "Found"

4\. If key < A[mid]

search the left half

5\. Otherwise

search the right half

### Why I selected Binary Search

I selected binary search because the database may contain one million or more records. Checking every record is not practical for every search.

Binary search removes approximately half of the remaining records after each comparison. Therefore, it is much faster than linear search for large sorted datasets.

It is also useful for this assignment because it is a recursive algorithm, so I can formulate and solve its recurrence relation in Tasks 6 and 7.

### 3. Hash Search

Hash search works differently from linear and binary search.

In hashing, the registration number is given to a hash function. The hash function produces an index or location where the record can be stored.

For example:

Registration Number

↓

Hash Function

↓

Index

↓

Student Record

If I want to search for a registration number, I calculate its hash value and go directly to the corresponding location.

### Type

Hashing/direct-access approach.

### Preprocessing

A hash table has to be created before searching.

Pseudocode

HashSearch(table, key)

1\. Calculate index = hash(key)

2\. Go to table[index]

3\. Check whether key is present

4\. If key is found

return "Found"

5\. Otherwise

return "Not Found"

### Collision

Sometimes two registration numbers may produce the same index. This is called a collision.

For example:

Key 1 → Index 5

Key 2 → Index 5

In this situation, a collision-handling technique is needed.

### Why I selected Hash Search

I selected hashing because it is a completely different searching strategy. It can provide very fast average searching when the hash function is good and collisions are controlled.

The disadvantage is that it requires additional memory and preprocessing.

**Overall Comparison**

Algorithm | Strategy | Sorted Data Required? | Preprocessing

Linear Search | Sequential | No | No

Recursive Binary Search | Divide and Conquer | Yes | Sorting

Hash Search | Hashing | No | Hash table

## Conclusion

I selected these three algorithms because they represent three different approaches to the same problem. Linear search is simple, binary search is efficient for sorted data, and hashing can provide very fast average searching.

This gives a good basis for comparing their efficiency as the number of student records increases.

## TASK 2 – IDENTIFY THE BASIC OPERATION

### Introduction

The basic operation is the operation that is performed repeatedly during the execution of an algorithm and contributes significantly to its running time.

By counting this operation, I can mathematically analyse how the algorithm behaves when the input size increases.

For this problem, the input size n represents the number of student records.

### 1. Basic Operation of Linear Search

The main operation in linear search is the comparison:

A[i] == key

This operation occurs inside the loop.

For example, if there are five records:

1020 1030 1040 1050 1060

and the required registration number is 1060, the algorithm performs:

1020 == 1060 → No

1030 == 1060 → No

1040 == 1060 → No

1050 == 1060 → No

1060 == 1060 → Yes

Therefore, five comparisons are performed.

If the number of records is n, the maximum number of comparisons is n.

So, the number of comparisons increases directly with n.

### 2. Basic Operation of Binary Search

The main operation in binary search is the comparison between the required key and the middle element.

A[mid] == key

There are also comparisons to decide whether the key is smaller or greater than the middle element.

The important point is that after every comparison, approximately half of the records are eliminated.

For example, if:

\\[ n=16 \\]

then the search sizes are approximately:

16 → 8 → 4 → 2 → 1

Therefore, only a small number of comparisons are needed.

The number of operations increases approximately as:

\\[ $\log_2$ n \\]

### 3. Basic Operation of Hash Search

For hash search, the main operations are:

index = hash(key)

and checking the corresponding location.

With a good hash function, the required record can usually be located directly.

Therefore, the average number of operations does not increase significantly when n increases.

However, if collisions occur, additional comparisons may be necessary.

Comparison

Algorithm | Basic Operation | Effect of Increasing n

Linear Search | Compare each record | Increases directly

Binary Search | Compare with middle | Increases logarithmically

Hash Search | Calculate index/check location | Approximately constant on average

Why these operations are suitable

These operations are suitable for efficiency analysis because they are closely related to the main work performed by each algorithm.

For linear search, the number of comparisons can become n.

For binary search, the number of comparisons depends on how many times the data can be divided by two.

For hashing, the main search operation normally remains approximately constant, although collisions can change the behaviour.

### Conclusion

By identifying the basic operation, I can use it to mathematically derive the time complexity of each searching algorithm in the next task.

TASK 3 – DERIVE THE TIME-COMPLEXITY FUNCTION MATHEMATICALLY

## Introduction

Time complexity describes how the running time of an algorithm changes when the input size n increases.

Here:

\\[ n=number of student records \\]

I will derive the running time by counting the basic operations.

## 1. Linear Search

Suppose there are n records.

The algorithm compares the key with each record.

In the worst case:

1st record → 1 comparison

2nd record → 1 comparison

3rd record → 1 comparison

nth record → 1 comparison

Therefore:

\\[ T(n)=1+1+1+\cdots+1 \\]

There are n terms.

Therefore:

\\[ T(n)=n \\]

Including some constant operations:

\\[ T(n)=an+b \\]

where a and b are constants.

Therefore:

\\[ T(n)=Θ(n) \\]

## Preprocessing cost

There is no preprocessing cost because linear search can work directly on the original records.

Therefore, total cost is:

\\[ T(n)=Θ(n) \\]

## 2. Binary Search

Binary search reduces the input size by half after every comparison.

Initially:

\\[ n \\]

After one step:

\\[ n/2 \\]

After two steps:

\\[ n/4 \\]

After three steps:

\\[ n/8 \\]

After k steps:

\\[ n/2^k \\]

The search finishes when only one element remains.

Therefore:

\\[ n/2^k=1 \\]

Multiplying both sides by \\(2^k\\):

\\[ n=2^k \\]

Taking logarithm:

\\[ k=\log_2 n \\]

Therefore:

\\[ T(n)=c\log_2 n+d \\]

where c and d are constants.

Hence:

\\[ T(n)=\Theta(\log n) \\]

## Preprocessing

Binary search requires sorted records.

If the records are initially unsorted, sorting is required.

Sorting generally requires:

\\[ O(n\log n) \\]

Therefore:

\\[ T_{total}(n) = O(nlog n)+O(log n) \\]

For one complete preparation and search:

\\[ T_{total}(n)=O(nlog n) \\]

However, after sorting, repeated searches only require logarithmic time each.

### 3. Hash Search

Creating a hash table requires processing the records.

If there are n records:

\\[ T_{preprocessing}(n)=O(n) \\]

After creating the table, an average search requires approximately constant time:

\\[ T_{search}(n)=O(1) \\]

Therefore, for one search including preprocessing:

\\[ T(n)=O(n)+O(1) \\]\\[ T(n)=O(n) \\]

But if the table has already been created and many searches are performed:

\\[ T_{each search}=O(1) \\]

Therefore, the preprocessing cost becomes worthwhile when there are many searches.

### Final Comparison

Algorithm | Search Complexity | Preprocessing

Linear Search | \\(Θ(n)\\) | None

Binary Search | \\(Θ(log n)\\) | \\(O(nlog n)\\) sorting

Hash Search | Average \\(O(1)\\) | \\(O(n)\\)

This mathematical analysis shows why the algorithms behave differently as the database becomes larger.

## TASK 4 – BEST, AVERAGE AND WORST-CASE BEHAVIOUR

The running time of an algorithm can change depending on the position of the required registration number.

Therefore, I consider three cases:

- Best case
- Average case
- Worst case

### 1. Linear Search

#### Best Case

The required registration number is the first record.

Example:

Key = 1001

1001  1002  1003  1004  1005

↑

Found

Only one comparison is required.

$$T(n)=1$$

Therefore:

$$Best=\Theta(1)$$

#### Average Case

The record is approximately in the middle.

For n records, approximately:

$$n/2$$

comparisons are required.

Therefore:

$$Average=\Theta(n)$$

#### Worst Case

The record is the last record or is not present.

Therefore:

$$T(n)=n$$

So:

$$ Worst=\Theta(n) $$

### 2. Binary Search

#### Best Case

The required registration number is the middle element.

Only one comparison is needed.

$$ Best=\Theta(1) $$

#### Average Case

The search continues through several divisions.

The input becomes:

$$ n,\frac n2,\frac n4,\frac n8,\ldots $$

Therefore:

$$ Average=\Theta(\log n) $$

#### Worst Case

The key is at the final possible position or does not exist.

The number of divisions is approximately:

$$ \log_2 n $$

Therefore:

$$ Worst=\Theta(\log n) $$

### 3. Hash Search

#### Best Case

The key is directly available at its calculated location.

$$ Best=\Theta(1) $$

#### Average Case

If the hash function distributes the registration numbers properly, the number of operations remains approximately constant.

$$ Average=\Theta(1) $$

#### Worst Case

If many registration numbers have the same hash value, collisions occur.

The algorithm may have to check many records.

Therefore:

\\[ Worst=Θ(n) \\]

### Comparison Table

Algorithm | Best | Average | Worst

Linear Search | \\(O(1)\\) | \\(O(n)\\) | \\(O(n)\\)

Binary Search | \\(O(1)\\) | \\(O(log n)\\) | \\(O(log n)\\)

Hash Search | \\(O(1)\\) | \\(O(1)\\) | \\(O(n)\\)

### Conclusion

The best case can be constant for all three methods. The important difference occurs in the average and worst cases.

Linear search can become very slow as the number of records increases. Binary search performs much better when the data is sorted. Hash search can be the fastest on average, but its performance depends on the hash function and collision handling.

**TASK 5 – PERFORM ASYMPTOTIC ANALYSIS**

Asymptotic analysis is used to study the growth of an algorithm when the input size becomes very large.

The main notations I use are:

- \\(O\\) — upper bound
- \\(Ω\\) — lower bound
- \\(Θ\\) — tight bound

Linear Search

For linear search:

\\[ T(n)=n \\]

Therefore:

\\[ O(n) \\]

and:

\\[ Ω(n) \\]

for the worst-case growth.

Hence:

\\[ Θ(n) \\]

In the worst case, if the number of records increases from 1,000 to 1,000,000, the number of comparisons can increase from approximately 1,000 to 1,000,000.

Therefore, linear growth becomes expensive for a very large database.

Binary Search

Binary search has:

\\[ T(n)=log₂n \\]

Therefore:

\\[ O(log n) \\]

and:

\\[ Θ(log n) \\]

For example:

\\[ log₂(1000)≈10 \\]\\[ log₂(10000)≈14 \\]\\[ log₂(100000)≈17 \\]\\[ log₂(1000000)≈20 \\]

This shows that increasing the database from 1,000 to 1,000,000 does not increase the number of binary-search comparisons very much.

Hash Search

For a well-designed hash table:

\\[ T(n)=O(1) \\]

on average.

Therefore:

\\[ Θ(1) \\]

is the average search complexity.

However, if many collisions occur:

\\[ O(n) \\]

can occur in the worst case.

Overall Growth

For average search behaviour:

\\[ O(1)\<O(log n)\<O(n) \\]

Therefore, for a large student database, hash search has the best average growth, followed by binary search and then linear search.

However, Big-O alone should not decide the final algorithm. Preprocessing, memory requirements, updates and actual experimental performance must also be considered.

## TASK 6 – FORMULATE THE RECURRENCE RELATION

For the recursive approach, I selected recursive binary search.

The recurrence relation should be obtained from the actual behaviour of the algorithm.

Suppose there are n records.

The algorithm first checks the middle record. This takes constant time.

After that, only one half of the records needs to be searched.

Therefore, the new problem size becomes:

\\[ \frac n2 \\]

There is only one recursive call, because binary search does not search both halves. It chooses only the half that may contain the required registration number.

The work outside the recursive call is constant.

Let this constant work be c.

Therefore:

\\[ T(n)=T(\frac n2)+c \\]

### Components of the recurrence

Original problem size:

\\[ n \\]

Recursive subproblem size:

\\[ \frac n2 \\]

Number of recursive calls:

\\[ 1 \\]

Work outside recursive call:

\\[ c \\]

Base case:

When there are no elements:

\\[ T(0)=c \\]

or when one element remains:

\\[ T(1)=c \\]

Therefore, the complete recurrence is:

\\[ T(n)=T(n/2)+c \\]

with:

\\[ T(1)=c \\]

This recurrence represents the actual working process of recursive binary search.

TASK 7 – SOLVE THE RECURRENCE RELATION

The recurrence obtained is:

\\[ T(n)=T(n/2)+c \\]

I will use the iteration method because the input size is repeatedly divided by 2.

This method is similar to the substitution/iteration approach studied in the notes.

### Step 1

Start with:

\\[ T(n)=T(n/2)+c \\]

Now substitute \\(n/2\\) for n:

\\[ T(n/2)=T(n/4)+c \\]

Therefore:

\\[ T(n)=T(n/4)+c+c \\]\\[ T(n)=T(n/4)+2c \\]

### Step 2

Again:

\\[ T(n/4)=T(n/8)+c \\]

Therefore:

\\[ T(n)=T(n/8)+3c \\]

### Step 3

Continuing the process:

\\[ T(n)=T(n/16)+4c \\]

The pattern is:

\\[ T(n)=T(n/2^k)+kc \\]

Step 4 – Reach Base Case

The recursion stops when the problem size becomes 1.

Therefore:

\\[ n/2^k=1 \\]

So:

\\[ n=2^k \\]

Taking logarithm on both sides:

\\[ $\log_2 n = k$ \\]

Therefore:

\\[ $k = \log_2 n$ \\]

Substitute into the recurrence:

\\[ $T(n) = T(1) + c\log_2 n$ \\]

Since \\(T(1)\\) and c are constants:

\\[ $T(n) = a + b\log_2 n$ \\]

Therefore:

\\[ $T(n) = \Theta(\log n)$ \\]

### Interpretation

The result means that the recursive binary search becomes only slightly slower when the database becomes much larger.

For example, a database of one million records needs only about:

\\[ $\log_2(1{,}000{,}000) \approx 20$ \\]

levels of searching.

This is why binary search is much more scalable than linear search.

## TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

For experimental testing, I used Python because it is simple to implement and allows the execution time to be measured.

The three algorithms are implemented separately so that their performance can be compared.

Start

sizes ← [1000, 10000, 100000, 1000000]

for each n in sizes:

	data ← n random unique registration numbers

	sorted_data ← sort(data)  // for binary search

	table ← hash table of data  // for hash search

	keys ← 20 random keys from data

	for each search method (Linear, Binary, Hash):

start timer

for each key in keys:

search(key) and count comparisons

stop timer

store average time and average comparisons

	print results for n

plot comparisons vs n

plot time vs n

End

### Test Conditions

I used four different input sizes:

\\[ 1000,\ 10000,\ 100000,\ 1000000 \\]

The same type of registration-number data is used for comparison.

The search key is selected as the last record so that linear search performs a large number of comparisons.

For binary search, the records are sorted.

For hash search, the records are stored in a hash table.

## Purpose of the experiment

The purpose is not just to prove that the programs work. The purpose is to check whether the experimental behaviour agrees with the theoretical complexity.

The assignment specifically requires the implementation to correspond to the algorithms analysed in the earlier tasks.

## TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

After running the programs, I record the execution time for each input size.

The blank values should be filled using the actual output obtained from my computer.

The assignment requires experimental measures such as execution time, number of comparisons or other suitable key operations.

I can also compare the number of comparisons.

From the comparison count, linear search increases directly with n, while binary search increases very slowly. Hash search remains approximately constant for normal cases.

A graph can be drawn with:

- X-axis → Number of records
- Y-axis → Execution time
- Three lines → Linear, Binary and Hash search

### Important

The actual execution times should be taken from the computer rather than assuming them. The assignment asks for collected experimental data and appropriate tables/graphs.

## TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

Theoretical analysis gives the expected growth of an algorithm, while experimental analysis gives the actual behaviour when the program is executed.

I compared the results of the three algorithms with their theoretical complexities.

Linear Search

The theoretical worst-case complexity is:

\\[ O(n) \\]

Therefore, if the number of records increases, the number of comparisons should also increase.

For example:

\\[ 1000→10000→100000→1000000 \\]

The number of possible comparisons also increases in the same direction.

Therefore, the experimental results should show an increasing execution time.

Binary Search

The theoretical complexity is:

\\[ O(\log n) \\]

The search area is divided by two after every comparison.

For one million records:

\\[ \log_2(1000000)\approx 20 \\]

Therefore, even though there are one million records, only around twenty levels of search are needed.

The experimental execution time should therefore increase very slowly as n increases.

Hash Search

The average theoretical complexity is:

\\[ O(1) \\]

Therefore, the search time should remain approximately constant when the number of records increases.

However, actual performance may change because of hash table construction, memory usage and collisions.

### Why Experimental Results May Differ

The theoretical complexity does not measure exact seconds.

Actual execution time depends on:

- Processor speed
- Available memory
- Programming language
- Python interpreter
- Operating system
- Background processes
- Data arrangement
- Hash collisions

Therefore, I should not expect the exact execution time to perfectly follow the mathematical values.

What is important is the growth pattern.

### Final comparison

If the experiment shows that:

- Linear search increases rapidly,
- Binary search increases slowly,
- Hash search remains approximately constant,

then the experimental results agree with the theoretical analysis.

This supports the mathematical analysis performed in the previous tasks.

**TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN**

After analysing the three approaches, I need to select the most suitable algorithm for the university student-record system.

Linear Search

Linear search has the advantage of being very simple.

It does not require sorting or preprocessing.

However, its worst-case complexity is:

\\[ O(n) \\]

For one million records, one search may require approximately one million comparisons.

If many users search the database, this can become expensive.

Therefore, I would not select linear search for the final large-scale system.

Binary Search

Binary search is much more efficient:

\\[ O(\log n) \\]

It is suitable for large sorted datasets.

However, the data must be sorted before searching.

If the database changes frequently, maintaining the sorted structure may also introduce additional work.

Therefore, binary search is a strong alternative, but it is not necessarily the best choice when the system has very frequent searches and updates must also be handled.

Hash Search

Hash search has an average search complexity of:

\\[ O(1) \\]

This means that the search operation can remain approximately constant even when the number of records becomes very large.

The main disadvantages are:

- Extra memory is required.
- A hash table must be created.
- Collisions can occur.
- Good hash-function design is important.

However, if the database is large and there are many searches, the preprocessing cost can be justified.

### Final Recommendation

I recommend Hash Search for the final system.

The main reason is that the system may contain one million or more records and may receive many search requests.

The hash table can be created once and reused for many searches.

Therefore, although preprocessing requires:

\\[ O(n) \\]

the individual search can have:

\\[ O(1) \\]

average complexity.

This is more suitable for a search-heavy student-record system.

### Final decision

\\[ Recommended approach = Hash Search \\]

The main trade-off is additional memory and collision handling, but the benefit of fast searching makes it suitable for the given application.

## TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

### Changed Requirement

The university now needs to process:

\\[ 50,000 searches per hour \\]

on:

\\[ 1,000,000 student records \\]

New student records are added only:

\\[ 1 time per day \\]

This changes the design consideration because the system is now performing a very large number of searches compared with the number of updates.

### A. Impact Analysis

Originally, the problem was to find an efficient search algorithm for a large database.

Now the system has a specific operating condition:

- One million records
- 50,000 searches every hour
- Very few updates

The search frequency is important because even a small difference in the cost of one search can become significant when the search is repeated thousands of times.

For example, if an algorithm requires a large number of comparisons for every search, the total work becomes very high.

The low update frequency is also important.

Since records are added only once per day, the system does not need to perform expensive preprocessing after every search.

Therefore, a preprocessing step can be performed when the database is updated and the resulting structure can be reused throughout the day.

### B. Evaluation of Original Design

My original recommendation was hash search.

I would retain this approach because the new requirement makes hashing even more suitable.

For hash search:

\\[ Average=O(1) \\]

Therefore, a large number of searches can be handled efficiently.

Compare this with linear search:

\\[ O(n) \\]

For one million records, a single worst-case search can require up to:

\\[ 1,000,000 \\]

comparisons.

If thousands of searches are performed, this becomes very expensive.

Binary search is better:

\\[ O(\log n) \\]

For one million records:

\\[ \log_2(1000000) \approx 20 \\]

So binary search is also efficient.

However, it requires sorted records.

Since the database receives new records only once per day, sorting is possible. Therefore, binary search is a valid alternative.

But hashing still provides approximately constant average search time.

### C. Adaptation or Redesign

I would modify and retain the original hash-based design rather than completely redesign it.

The database can maintain a hash table containing:

Registration Number → Student Record

When new students are added once per day, the hash table can be updated.

The process becomes:

New student records

↓

Daily update

↓

Update hash table

↓

50,000 search requests/hour

↓

Hash lookup

↓

Student record

The initial construction of the hash table takes approximately:

\\[ O(n) \\]

If new records are inserted, the table can be updated instead of performing the complete search structure construction for every query.

The average search remains:

\\[ O(1) \\]

### D. Evidence-Based Defence

I would retain the hash-based approach because the new requirement has many searches but very few updates.

For one million records:

Linear search → up to 1,000,000 comparisons per search in the worst case.

1.

Binary search → approximately $\log_2(1{,}000{,}000) \approx 20$ comparisons.

1.

Hash search → approximately constant-time lookup on average.

There are:

[50,000]

searches every hour, so the repeated search cost is much more important than the once-per-day preprocessing cost.

Therefore, even though creating the hash table requires additional work and memory, this cost is performed infrequently and can be reused for many searches.

FINAL CONCLUSION

Through this assignment, I understood that selecting an algorithm is not only about finding the smallest Big-O value. We also need to consider preprocessing, input size, memory, number of searches and number of updates.

Linear search is easy to implement but becomes inefficient when the number of records becomes very large.

Binary search is much faster because it reduces the search space by half. However, it requires sorted data.

Hash search requires preprocessing and additional memory, but its average search time is approximately constant.

For the final requirement of one million records, 50,000 searches per hour and only daily updates, I selected a hash-based search system with daily updates.

This design is suitable because the preprocessing cost can be reused for a very large number of search operations.

**Code of Conduct Statement:** I confirm that the work submitted for this assignment is prepared for academic purposes and that experimental results should be recorded honestly from the actual program execution.

**Appendix – Simple Python Implementation**

```
import time
import matplotlib.pyplot as plt
def linear_search(a, key):
    for i in range(len(a)):
        if a[i] == key:
            return i
    return -1
def binary_search(a, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if a[mid] == key:
        return mid
    elif key < a[mid]:
        return binary_search(a, low, mid - 1, key)
    else:
        return binary_search(a, mid + 1, high, key)
sizes = [1000, 10000, 100000, 1000000]
linear_times = []
binary_times = []
```

```
hash_times = []
results = []
for n in sizes:
    records = []
    for i in range(n):
        records.append(100000 + i)
    key = 100000 + n // 2
    start = time.perf_counter()
    p1 = linear_search(records, key)
    end = time.perf_counter()
    linear_time = (end - start) * 1000000
    linear_times.append(linear_time)
    sorted_records = sorted(records)
    start = time.perf_counter()
    p2 = binary_search(
        sorted_records,
        0,
        len(sorted_records) - 1,
        key
    )
    end = time.perf_counter()
    binary_time = (end - start) * 1000000
    binary_times.append(binary_time)
    table = {}
    for i in range(n):
        table[records[i]] = i
    start = time.perf_counter()
    p3 = table.get(key, -1)
    end = time.perf_counter()
    hash_time = (end - start) * 1000000
    hash_times.append(hash_time)
    results.append([
        n,
        linear_time,
        binary_time,
        hash_time
    ])
```

```
print("\nTASK 9 - EXPERIMENTAL PERFORMANCE")
print("-" * 75)
print(
    f"{'Records':<15}"
    f"{'Linear (µs)':<20}"
    f"{'Binary (µs)':<20}"
    f"{'Hash (µs)':<20}"
)
print("-" * 75)
for row in results:
    print(
        f"{row[0]:<15}"
        f"{row[1]:<20.3f}"
        f"{row[2]:<20.3f}"
        f"{row[3]:<20.3f}"
    )
with open("search_results.csv", "w") as file:
    file.write(
        "Records,Linear Search Time (microseconds),"
        "Binary Search Time (microseconds),"
        "Hash Search Time (microseconds)\n"
    )
    for row in results:
        file.write(
            f"{row[0]},{row[1]},{row[2]},{row[3]}\n"
        )
print("\nResults saved as: search_results.csv")
plt.figure(figsize=(9, 6))
plt.plot(
    sizes,
    linear_times,
    marker="o",
    label="Linear Search"
)
plt.plot(
    sizes,
    binary_times,
```

```
    marker="o",
    label="Binary Search"
)
plt.plot(
    sizes,
    hash_times,
    marker="o",
    label="Hash Search"
)
plt.xlabel("Number of Records")
plt.ylabel("Execution Time (microseconds)")
plt.title("Search Algorithm Performance")
plt.xscale("log")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("search_performance_graph.png", dpi=300)
plt.show()
print("Graph saved as: search_performance_graph.png")
```

```
TASK 9 - EXPERIMENTAL PERFORMANCE
------------------------------------------------------------------------
Records        Linear (µs)         Binary (µs)         Hash (µs)
------------------------------------------------------------------------
1000           36.884              11.356              1.969
10000          477.231             20.228              6.427
100000         6155.681            30.516              7.030
1000000        46718.918           30.216              7.759

Results saved as: search_results.csv
```

